Lecture 5 : TinyML et optimisation des modèles TensorFlow Lite

Quantification, élagage, distillation et architectures légères, projet EcoTri 4.0

Le modèle fait 3 Mo et la Portenta n'a que 1 Mo de RAM. Que faire ?

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

Mesurer le coût | Architectures légères | Quantification | Élagage et distillation | Chaîne TFLite et Pareto

- **Mesurer le coût** : *Comment savoir si un modèle tient, avant même de le déployer ?*
- **Architectures légères** : *Comment concevoir un réseau qui coûte peu par construction ?*
- **Quantification** : *Comment passer de 32 bits à 8 bits sans perdre la précision ?*
- **Élagage et distillation** : *Peut-on retirer des poids ou apprendre d'un grand modèle ?*
- **Chaîne TFLite et Pareto** : *Comment produire le `.tflite` et choisir le bon compromis (jalon J5) ?*

Objectifs

Ce que vous saurez faire en sortant de la salle

À la fin de cette lecture, vous serez capables de :

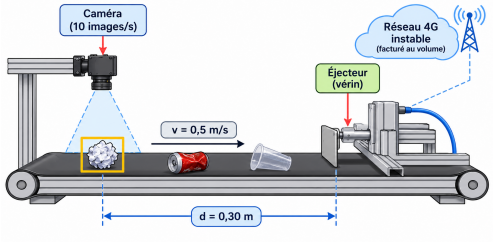
- **Calculer** le coût d'un modèle : paramètres, MAC, taille en Flash et **pic de RAM** des activations.
- **Expliquer** les convolutions séparables, le multiplicateur de largeur α et l'effet de la résolution d'entrée.
- **Dériver** la quantification affine INT8 (s, z) et l'arithmétique entière qui en découle.
- **Mettre en œuvre** avec TensorFlow Lite (LiteRT) : plage dynamique, float16, INT8 complet, QAT, élagage, distillation.
- **Déployer** un `.tflite` INT8 sur la Portenta H7 (OpenMV) et mesurer latence et mémoire.
- **Argumenter** un choix de modèle à partir d'un **front de Pareto** précision / latence / mémoire.

Le fil rouge et le livrable

- SP5 « Reconnaître le matériau » : classer plastique, verre, carton, métal sur la Portenta, en moins de 600 ms.
- **Jalon J5** : rapport d'optimisation comparant au moins trois modèles, et choix argumenté (comparé à la référence à règles J2).

Le problème déclencheur

Un bon modèle… qui ne tient pas



La situation

- L'équipe a entraîné sur Colab un MobileNetV2 ($\alpha = 1$, 224×224) : **91 %** de précision sur TrashNet.
- Fichier `.tflite` : **8,9 Mo** en float32 ; pic d'activations ≈ **4,8 Mo**.
- La Portenta : 2 Mo de Flash interne, **1 Mo** de SRAM, 8 Mo de SDRAM partagée avec l'image.

À vous (5 min, par équipe)

Listez **trois leviers** pour faire tenir le classifieur. Pour chacun, estimez le **facteur de gain** (mémoire ou calcul) et ce que l'on risque de **perdre**.

Mesurer le coût

Comment savoir si un modèle tient, avant même de le déployer ?

Mesurer le coût

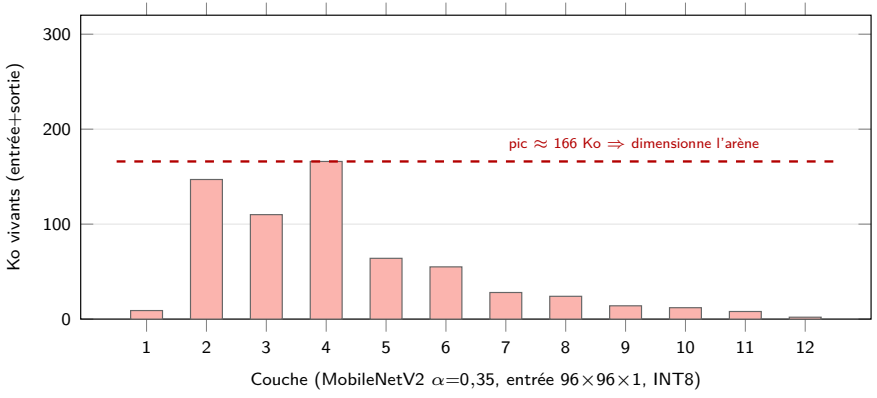
Quatre métriques, quatre ressources différentes

Métrique	Formule (couche conv $k \times k$)	Ressource	Limite Portenta
Paramètres P	$k^2 C_{in} C_{out} + C_{out}$	Flash	2 Mo (interne)
Taille du modèle	$P \times$ octets par poids	Flash	$\approx$ 1 Mo réservé
MAC	$H_{out} W_{out} k^2 C_{in} C_{out}$	temps, énergie	600 ms au total
Pic d'activations	$\max_\ell$ (entrées + sorties vivantes)	**SRAM**	$\approx$ 300–400 Ko utiles

! les **poids** se payent en Flash, une seule fois ; les **activations** se payent en SRAM, et seul leur **pic** compte, car le moteur réutilise l'arène couche après couche.

Mesurer le coût

Le pic d'activations : souvent le vrai verrou (Lin et al., MIT, MCUNet 2020)



Lecture : le pic arrive dans les **premières** couches, à haute résolution, sur les blocs à forte expansion. Réduire la résolution d'entrée ou les canaux de ces couches fait baisser le pic ; quantifier divise toutes les barres par 4.

Couche 1×1 d'expansion, entrée $48 \times 48 \times 16$, sortie $48 \times 48 \times 96$

- Paramètres : $16 \times 96 + 96 = 1\,632$.
- MAC : $48 \times 48 \times 16 \times 96 \approx$ **3,5 M**.
- Activations : entrée 36,9 Ko + sortie **221 Ko** en INT8, soit 1 Mo en float32.

Conclusion

Une couche qui ne pèse que **1,6 Ko** de poids exige **258 Ko** de SRAM en INT8 : la taille du fichier `.tflite` ne dit rien de la RAM nécessaire.

> **!** Edge Impulse, STM32Cube.AI ou le profileur TFLite Micro affichent ce pic ; au TP, on le mesure via `gc.mem_free()` et la taille d'arène.

Question flash

?

Un CNN a une première couche conv 3×3, 8 filtres, stride 1, sur une entrée $96 \times 96 \times 1$ (padding *same*). Donnez ses paramètres, ses MAC et la taille de sa sortie en INT8 et en float32.

Question flash

? Un CNN a une première couche conv 3×3, 8 filtres, stride 1, sur une entrée $96 \times 96 \times 1$ (padding *same*). Donnez ses paramètres, ses MAC et la taille de sa sortie en INT8 et en float32.

Réponse

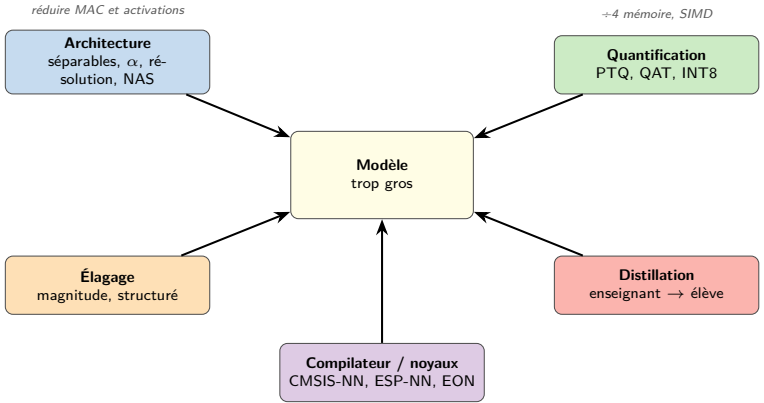
Paramètres : $3 \times 3 \times 1 \times 8 + 8 = 80$. MAC : $96 \times 96 \times 9 \times 8 \approx 663$ k. Sortie : $96 \times 96 \times 8 = 73{,}7$ Ko en INT8, **295 Ko** en float32 : déjà la moitié de la SRAM utile, pour une couche de 80 paramètres ! D'où le **stride 2** dès la première couche dans les réseaux mobiles.

Architectures légères

Comment concevoir un réseau qui coûte peu par construction ?

Architectures légères

La boîte à outils de l'optimisation

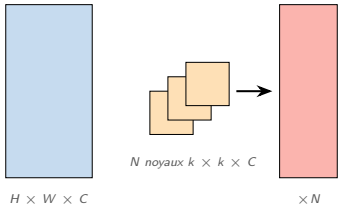


On agit d'abord sur l'**architecture** (le plus gros gain), puis on **compresse** le modèle choisi.

Architectures légères

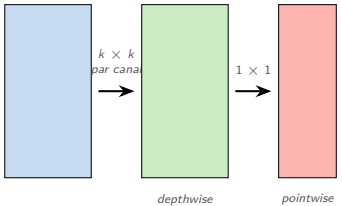
Convolution séparable en profondeur (Howard et al., Google, MobileNet 2017)

Convolution standard



MAC = $HW\,k^2\,C\,N$

Séparable en profondeur (MobileNet)



MAC = $HW\,(k^2C + CN)$

rapport = $\frac{1}{N} + \frac{1}{k^2}$: pour k=3, N=64 $\Rightarrow \approx 8\times$ moins de calcul

On sépare le filtrage **spatial** (un noyau $k \times k$ par canal) du mélange des **canaux** (convolution 1×1). MobileNetV2 (Sandler et al., 2018) ajoute les **blocs résiduels inversés** : expansion 1×1, depthwise 3×3, projection linéaire.

Architectures légères

Les trois boutons de MobileNet : largeur, résolution, profondeur

- **Largeur** α : multiplie le nombre de canaux. Paramètres et MAC $\propto \alpha^2$.
- **Résolution** ρ : MAC et activations $\propto \rho^2$; les paramètres ne changent pas.
- **Profondeur** : retirer des blocs (EfficientNet, Tan & Le 2019, règle les trois à la fois).
- **FOMO** (Edge Impulse) : MobileNetV2 tronqué qui sort une carte de centroïdes 12×12 pour la détection.

MobileNetV2	Entrée	MAC	Param.
$\alpha = 1$	224	300 M	3,4 M
$\alpha = 0{,}35$	224	59 M	1,7 M
$\alpha = 0{,}35$	96	**11 M**	1,7 M
$\alpha = 0{,}35$, sans tête	96	11 M	**0,41 M**

Valeurs Keras Applications ; la tête 1000 classes (1280×1000) est remplacée par 4 classes.



on part des poids ImageNet d'un MobileNetV2 $\alpha = 0{,}35$, on remplace la tête par 4 classes et on affine : quelques centaines d'images par classe suffisent.

Architectures légères

Recherche d'architecture neuronale (NAS) : MCUNet (MIT HAN Lab)

- **TinyNAS** : réduit d'abord l'espace de recherche (résolution, largeur) pour qu'il respecte la Flash et la SRAM de la cible, puis cherche le meilleur réseau dedans.
- **TinyEngine** : moteur d'inférence co-conçu (planification mémoire, fusion d'opérateurs).
- MCUNetV2 (2021) : inférence **par patchs** des premières couches pour écraser le pic de RAM.

Résultat marquant

ImageNet à ≈ 70 % de précision top-1 sur un microcontrôleur commercial, avec moins de 1 Mo de Flash et quelques centaines de Ko de SRAM.

> **!** pas de NAS au TP ; on retient l'**idée** : la contrainte mémoire de la cible doit guider le choix de l'architecture **dès le départ**.

Question flash

?

Une couche conv 3×3 de 64 vers 64 canaux sur 24×24 coûte combien de MAC ? Et sa version séparable ? Quel est le gain ?

Question flash

? Une couche conv 3×3 de 64 vers 64 canaux sur 24×24 coûte combien de MAC ? Et sa version séparable ? Quel est le gain ?

Réponse

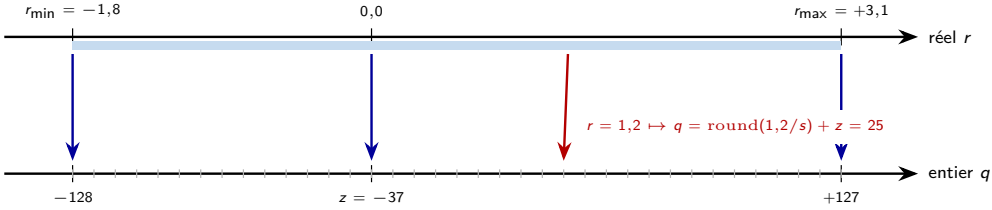
Standard : $24^2 \times 9 \times 64 \times 64 \approx 21{,}2$ M. Séparable : $24^2 \times (9 \times 64 + 64 \times 64) = 24^2 \times 4\,672 \approx 2{,}7$ M. Gain $\approx \mathbf{7{,}9}\times$, cohérent avec $1/N + 1/k^2 = 1/64 + 1/9 \approx 0{,}127$.

Quantification

Comment passer de 32 bits à 8 bits sans perdre la précision ?

Quantification

Le schéma affine (Jacob et al., Google, CVPR 2018)



$$q = \text{clamp}\left(\text{round}\left(\frac{r}{s}\right) + z,\ -128,\ 127\right) \qquad r \approx s\,(q - z) \qquad s = \frac{r_{\max} - r_{\min}}{255} \approx 0{,}0192$$

s (*scale*) : pas de quantification, réel ; z (*zero-point*) : entier tel que $r = 0$ soit représenté **exactement** (padding, ReLU).

Erreur d'arrondi bornée par $s/2$: plus la plage $[r_{\min}, r_{\max}]$ est large, plus l'erreur est grande. D'où l'importance de la **calibration**.

Quantification

La spécification INT8 de TensorFlow Lite

Tenseur	Représentation	Pourquoi
Poids	INT8 **symétrique** ($z = 0$), **par canal**	un s par filtre : les filtres de dynamiques différentes gardent leur précision
Activations	INT8 **asymétrique**, par tenseur	après ReLU, les valeurs sont ≥ 0 : on utilise toute la plage
Biais	INT32, $s_b = s_x \cdot s_w$, $z = 0$	s'ajoute directement à l'accumulateur 32 bits
Accumulateur	INT32	une somme de produits 8×8 bits dépasse 16 bits

! les plages des **poids** sont connues ; celles des **activations** ne le sont pas : le convertisseur fait passer un **jeu représentatif** (100 à 500 images réelles) pour relever r_{min} et r_{max} de chaque tenseur.

Quantification

Pourquoi le calcul reste entier : la dérivation

Produit scalaire réel $r_3 = \sum_i r_1^{(i)} r_2^{(i)}$, avec $r = s(q - z)$:

$$s_3(q_3 - z_3) = s_1 s_2 \sum_i (q_1^{(i)} - z_1)(q_2^{(i)} - z_2)$$

$$q_3 = z_3 + \underbrace{\frac{s_1 s_2}{s_3}}_{M} \underbrace{\sum_i (q_1^{(i)} - z_1)\, q_2^{(i)}}_{\text{INT32}} \qquad (z_2 = 0)$$

On écrit $M = 2^{-n} M_0$ avec $M_0 \in [0{,}5; 1)$ stocké en entier 32 bits : **une multiplication entière et un décalage**.

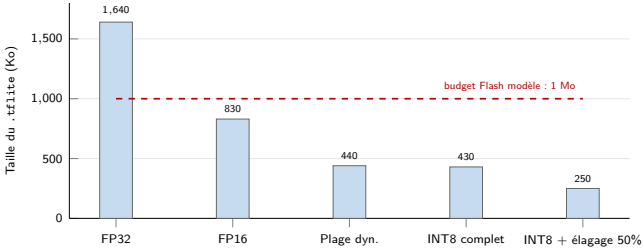
Conséquences

- Aucun flottant pendant l'inférence : idéal pour les MCU sans FPU et pour le SIMD (`SMLAD` : 2 MAC/cycle).
- ReLU6 et biais sont **fusionnés** dans la requantification.
- Mémoire ÷4, bande passante mémoire ÷4, énergie par MAC $\approx$ ÷18 (Horowitz, 2014).

Quantification

Les quatre recettes du convertisseur TensorFlow Lite (LiteRT)

Recette	Poids / activations	Taille	Données requises	Cible
float32 (aucune)	FP32 / FP32	$\times 1$	aucune	référence
float16	FP16 / FP32	$\times 1/2$	aucune	GPU mobile
Plage dynamique	INT8 / FP32 (à la volée)	$\times 1/4$	aucune	CPU mobile
INT8 complet	**INT8 / INT8**	$\times$**1/4**	**jeu représentatif**	**MCU**, NPU, Edge TPU



Quantification

INT8 complet en Python

```python
import tensorflow as tf

def rep_data():                        # 200 images réelles
    for x, _ in train_ds.unbatch().take(200):
        yield [tf.expand_dims(x, 0)]

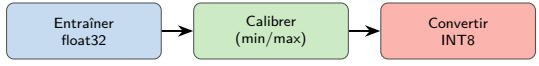
conv = tf.lite.TFLiteConverter.from_keras_model(model)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
conv.representative_dataset = rep_data
conv.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS_INT8]  # erreur si op non quantifiable
conv.inference_input_type  = tf.int8
conv.inference_output_type = tf.int8
open("trash_int8.tflite", "wb").write(conv.convert())
```

- Sans `supported_ops`, une opération non quantifiable reste silencieusement en float : le modèle ne tournera pas sur MCU.
- Entrée et sortie en `int8` : il faut **quantifier l'image** avec s, z de l'entrée.
- Depuis 2024, TensorFlow Lite s'appelle **LiteRT** ; l'API `tf.lite` reste disponible.

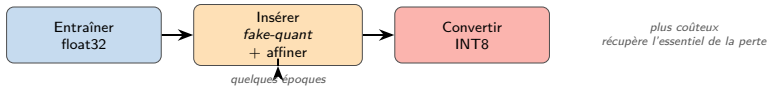
Quantification

PTQ ou QAT ?

Quantification post-entraînement (PTQ)



Entraînement conscient de la quantification (QAT)




```python
import tensorflow_model_optimization as tfmot
qat = tfmot.quantization.keras.quantize_model(model)
qat.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
            loss="sparse_categorical_crossentropy",
            metrics=["accuracy"])
qat.fit(train_ds, validation_data=val_ds, epochs=3)
```

Les nœuds *fake-quant* arrondissent en avant et laissent passer le gradient en arrière (*straight-through estimator*) : le réseau apprend à être robuste à l'arrondi.

Question flash

?

Les activations d'une couche, mesurées sur le jeu représentatif, vont de 0 à 6 (ReLU6). Calculez s et z en INT8, puis la valeur entière de $r = 2{,}5$. Quelle est l'erreur maximale ?

Question flash

?

Les activations d'une couche, mesurées sur le jeu représentatif, vont de 0 à 6 (ReLU6). Calculez s et z en INT8, puis la valeur entière de $r = 2{,}5$. Quelle est l'erreur maximale ?

Réponse

$s = 6/255 \approx 0{,}02353$; $r = 0$ doit donner $q = -128$, donc $z = -128$. $q = \mathrm{round}(2{,}5/0{,}02353) - 128 = 106 - 128 = -22$. Retour : $0{,}02353 \times (-22 + 128) = 2{,}494$. Erreur maximale $s/2 \approx$ **0,012**.

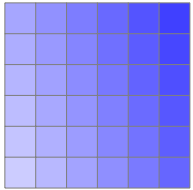
Élagage et distillation

Peut-on retirer des poids ou apprendre d'un grand modèle ?

Élagage et distillation

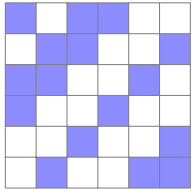
L'élagage : retirer ce qui ne sert pas (Han et al., Stanford, Deep Compression 2016)

Dense



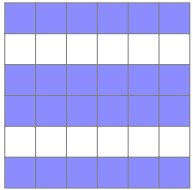
36 poids

Non structuré (50 %)



*zéros dispersés : petit après compression, **pas plus rapide** sur MCU*

Structuré (2 filtres sur 6)



*lignes entières retirées : couche plus petite, **gain réel** de MAC et de RAM*

Par magnitude : on met à zéro les poids de plus petite valeur absolue, progressivement (sparsité 0 % → 50–80 %), en ré-entraînant entre deux étapes.

Piège sur MCU

TFLite Micro calcule en **dense** : des zéros dispersés réduisent la taille compressée (zip, OTA) mais **pas** la latence. Seul l'élagage **structuré** accélère.

Élagage et distillation

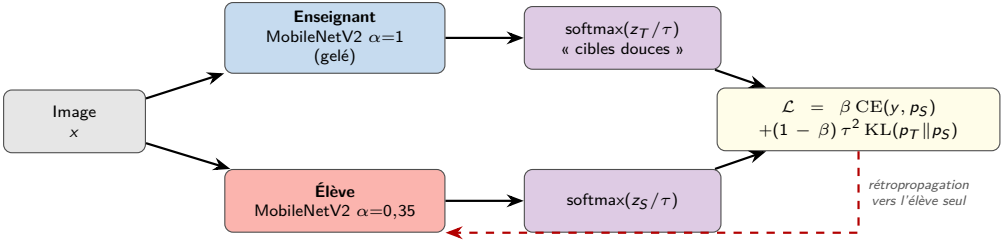
Élagage avec TensorFlow Model Optimization

```
import tensorflow_model_optimization as tfmot
pr = tfmot.sparsity.keras
sched = pr.PolynomialDecay(initial_sparsity=0.0,
          final_sparsity=0.5, begin_step=0,
          end_step=steps_par_epoque * 4)
m = pr.prune_low_magnitude(model, pruning_schedule=sched)
m.compile(optimizer="adam",
          loss="sparse_categorical_crossentropy",
          metrics=["accuracy"])
m.fit(train_ds, epochs=4,
      callbacks=[pr.UpdatePruningStep()])
m = pr.strip_pruning(m)      # retire les masques
# puis conversion INT8 comme précédemment
```

- Toujours **élaguer, puis quantifier** : la combinaison est cumulative (Deep Compression : 35 à 49× sur AlexNet et VGG).
- Vérifier la sparsité obtenue : `np.mean(w == 0)`.
- Mesurer la taille **zippée** pour voir le gain.

Élagage et distillation

La distillation des connaissances (Hinton, Vinyals, Dean, 2015)



La **température** $\tau > 1$ adoucit les probabilités : l'élève apprend que « verre » ressemble plus à « plastique » qu'à « carton ». Cette information est absente des étiquettes dures.

! $\tau \in [2,5]$, $\beta \approx 0{,}1$–$0{,}5$; gain de 1 à 3 points pour un élève très petit, à combiner avec la quantification.

Question flash

?

Votre modèle INT8 fait 430 Ko et tourne en 180 ms. Vous élaguez 50 % des poids (non structuré). Que deviennent la taille du `.tflite`, la taille zippée et la latence sur la Portenta ?

Question flash

? Votre modèle INT8 fait 430 Ko et tourne en 180 ms. Vous élaguez 50 % des poids (non structuré). Que deviennent la taille du `.tflite`, la taille zippée et la latence sur la Portenta ?

Réponse

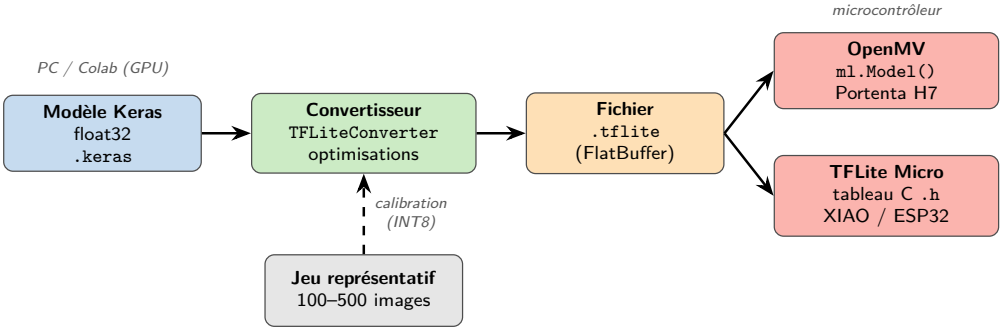
`.tflite` : $\approx$ **inchangé** (les zéros sont stockés). Zippé : nettement plus petit ($\approx \div 1{,}6$ à $\div 2$), utile pour les mises à jour OTA. Latence : **inchangée**, le noyau dense multiplie aussi les zéros. Pour gagner en temps : élagage **structuré** (filtres) ou réduire α.

Chaîne TFLite et Pareto

Comment produire le .tflite et choisir le bon compromis (jalon J5) ?

Chaîne TFLite et Pareto

Du notebook au microcontrôleur

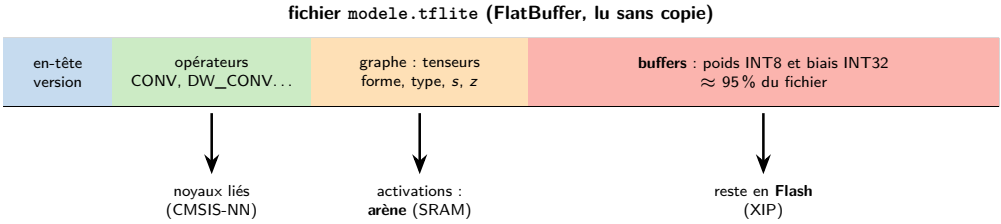


OpenMV (Portenta)

- On copie le `.tflite` et `labels.txt` sur le disque USB de la carte.
- Le firmware embarque TFLite Micro + CMSIS-NN.

Arduino / ESP-IDF (XIAO, ESP32)

- `xxd -i modele.tflite > modele.h` : le modèle devient un tableau C en Flash.
- On déclare une **arène** statique et les seuls opérateurs utiles (`MicroMutableOpResolver`).



- Format **FlatBuffer** : lisible directement en mémoire, sans désérialisation ni copie, donc exécutable depuis la Flash.
- Ouvrez-le avec **Netron** (netron.app) : vous y lisez les opérateurs, les formes, les types et les paramètres s, z de chaque tenseur.
- L'interpréteur TFLite Micro planifie l'arène à l'initialisation : si `AllocateTensors()` échoue, l'arène est trop petite.

Chaîne TFLite et Pareto

Inférence sur la Portenta avec OpenMV

```
import sensor, time, ml, gc

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.set_windowing((240, 240))
sensor.skip_frames(time=1500)

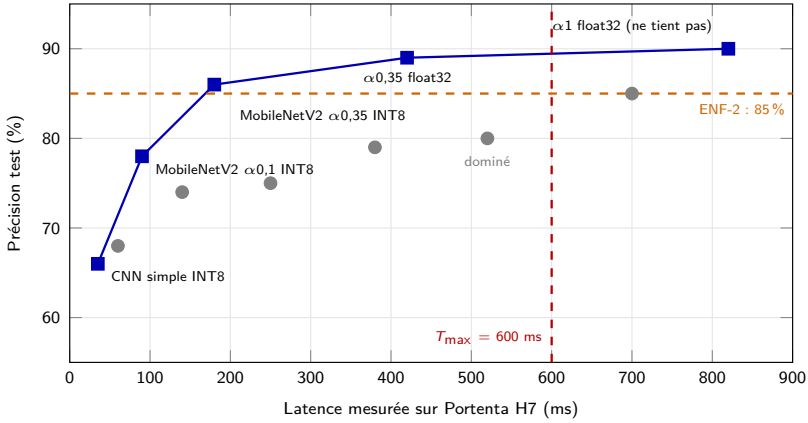
net = ml.Model("trash_int8.tflite", load_to_fb=True)
labels = [l.strip() for l in open("labels.txt")]
clock = time.clock()
while True:
    clock.tick()
    img = sensor.snapshot()
    t0 = time.ticks_us()
    p = net.predict([img])[0].flatten().tolist()
    dt = time.ticks_diff(time.ticks_us(), t0) / 1000
    k = p.index(max(p))
    print("%s %.2f | %.1f ms | tas %d" % (labels[k], p[k], dt, gc.
      mem_free()))
```

- `ml.Model` (OpenMV $\geq$ 4.5) redimensionne l'image à la taille d'entrée et la quantifie avec s, z du modèle.
- `load_to_fb=True` place le modèle dans le **frame buffer** (SDRAM) plutôt que dans le tas.
- Firmware plus ancien : `tf.load()` et `net.classify(img)`.

> **!** on chronomètre **l'inférence seule** et toute la boucle : c'est T_{inf} du budget de la lecture 1.

Chaîne TFLite et Pareto

Choisir : le front de Pareto



Un modèle est **dominé** s'il existe un autre modèle à la fois plus précis et plus rapide. On choisit sur le **front** (courbe bleue), dans la zone qui respecte les exigences ENF-1 (latence) et ENF-2 (précision), et on vérifie la mémoire.

Mesurer juste

- Évaluer le `.tflite` **quantifié** avec l'interpréteur, pas le modèle Keras.
- Même **prétraitement** sur PC et sur cible (gris, 96×96, normalisation).
- Évaluer sur les **images de la station**, pas seulement sur TrashNet (changement de domaine).
- Latence : médiane et **pire cas** sur au moins 100 inférences.

Quand la quantification fait mal

- Activations à très large plage (pas de BatchNorm, pas de ReLU6) : s grand, erreur grande.
- Jeu représentatif trop petit ou non représentatif (que des images sombres).
- Couches de très petite taille (depthwise) : préférer la quantification **par canal**, puis la QAT.

! comparer couche par couche les sorties float et INT8 (outil *Quantization Debugger* de TFLite) pour trouver la couche fautive.

Question flash

?

Trois candidats pour EcoTri : (A) CNN simple INT8, 72 %, 40 ms, 60 Ko d'arène ; (B) MobileNetV2 α0,35 INT8, 87 %, 180 ms, 170 Ko ; (C) MobileNetV2 α0,35 float32, 88 %, 430 ms, 680 Ko. Exigences : 85 %, $T_{\text{inf}} \leq 250$ ms, arène ≤ 300 Ko. Lequel ? Et si le règle-à-seuils J2 atteignait 80 % ?

Question flash

?

Trois candidats pour EcoTri : (A) CNN simple INT8, 72 %, 40 ms, 60 Ko d'arène ; (B) MobileNetV2 α0,35 INT8, 87 %, 180 ms, 170 Ko ; (C) MobileNetV2 α0,35 float32, 88 %, 430 ms, 680 Ko. Exigences : 85 %, $T_{\text{inf}} \leq 250$ ms, arène ≤ 300 Ko. Lequel ? Et si le règle-à-seuils J2 atteignait 80 % ?

Réponse

(A) rate la précision ; (C) dépasse latence et mémoire pour 1 point de gain. **(B)** est le seul admissible. Même face à J2 à 80 %, (B) se justifie : $+7$ points, et la règle-à-seuils est fragile à l'éclairage (lecture 2). (A) pourrait servir de **premier étage** d'une cascade (lecture 6).

À retenir

- **Quatre coûts** : paramètres (Flash), MAC (temps, énergie), taille du fichier, et surtout **pic d'activations** (SRAM).
- **L'architecture d'abord** : séparables, α, résolution d'entrée ; le transfer learning rend les petits modèles utiles.
- **INT8 complet** : $r = s(q - z)$, poids symétriques par canal, activations calibrées sur un jeu représentatif, calcul 100 % entier ; $\div 4$ en mémoire.
- **QAT** quand la PTQ perd trop ; **élagage structuré** pour gagner du temps ; **distillation** pour aider un très petit élève.
- **Mesurer sur la cible** le `.tflite` quantifié, puis choisir sur le **front de Pareto** sous les contraintes du cahier des charges.

Glossaire et références

Terme	Définition courte
PTQ	quantification après entraînement
QAT	entraînement conscient de la quantification
s, z	échelle et point zéro
Arène	zone SRAM des tenseurs (TFLM)
Sparsité	proportion de poids nuls
α	multiplicateur de largeur
LiteRT	nouveau nom de TFLite (2024)

- B. Jacob et al. (Google), « Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference », CVPR 2018.
- R. Krishnamoorthi, « Quantizing deep convolutional networks for efficient inference : A whitepaper », arXiv :1806.08342, 2018.
- S. Han, H. Mao, W. Dally (Stanford), « Deep Compression », ICLR 2016.
- G. Hinton, O. Vinyals, J. Dean, « Distilling the Knowledge in a Neural Network », 2015.
- A. Howard et al., « MobileNets », 2017 ; M. Sandler et al., « MobileNetV2 », CVPR 2018.
- J. Lin et al. (MIT), « MCUNet », NeurIPS 2020 ; cours MIT 6.5940 *TinyML and Efficient Deep Learning* (S. Han).
- V. Janapa Reddi (Harvard), *Machine Learning Systems*, chap. « Model Optimizations », mlsysbook.ai ; HarvardX TinyML.
- P. Warden, D. Situnayake, *TinyML*, O'Reilly, 2019 ; doc. LiteRT (ai.google.dev/edge/litert), OpenMV.

Prochaine étape

Tutorial 5 : de Keras au `.tflite` INT8 sur la Portenta. **TP 5** : comparaison de trois modèles, front de Pareto et jalon J5.